LOCK-BASED VS LOCK-FREE QUEUE PERFORMANCE

Technical Report

Saleh Yahya
Department of Computer Science
Lewis University
Chicago, IL 60462
salehyahya10.20@gmail.com

Christopher Fawaz
Department of Computer Science
Lewis University
Chicago, IL 60563
chrisfawaz53@gmail.com

ABSTRACT

This paper examines the performance differences between lock based and lock free queue implementations in a multi threaded environment. The purpose of this study is to evaluate how different synchronization strategies affect throughput, latency, and scalability under concurrent workloads. A lock based queue using mutexes and condition variables was compared with a lock free queue implemented using atomic operations and compare and swap CAS. Both implementations were benchmarked under varying levels of concurrency, including multiple producer and consumer thread configurations. The results showed that the lock based queue maintained stable and consistent performance across all tests, achieving higher throughput and lower latency in most scenarios. In contrast, the lock free queue underperformed and initially exhibited instability due to unsafe memory access during concurrent execution. After applying a temporary fix to prevent premature memory deallocation, the lock free implementation completed all tests but still did not outperform the lock based approach. These findings suggest that while lock free data structures offer theoretical advantages, their practical performance depends heavily on correct implementation and system level considerations. The study highlights the importance of empirical testing when evaluating concurrent system designs.

Introduction

Concurrent systems rely on efficient data structures to manage shared resources across multiple threads. As hardware continues to scale with increasing core counts, synchronization strategies play a critical role in determining system performance. This paper explores the differences between lock based and lock free queue implementations, focusing on how each approach affects throughput, latency, and scalability under concurrent workloads.

Theory

Concurrent data structures are designed to allow multiple threads to safely access shared resources. Two primary synchronization approaches are lock based and lock free techniques. Lock based synchronization uses mutual exclusion mechanisms such as mutexes to ensure that only one thread accesses critical sections at a time. While this guarantees correctness, it can introduce performance bottlenecks due to blocking and contention between threads. Lock free synchronization relies on atomic operations such as compare and swap to coordinate access without using locks. This approach allows threads to proceed without blocking, improving potential scalability. However, it introduces challenges including memory ordering, the ABA problem, and safe memory management. In theory, lock free data structures are expected to outperform lock based implementations under high contention. However, their performance depends heavily on correct implementation and system level behavior.

Lock-Based Queue Design

The lock based queue was implemented using traditional synchronization primitives. A mutex ensures mutual exclusion, while a condition variable prevents busy waiting by suspending consumer threads when the queue is empty. The queue follows a first in first out structure using a standard

container. This approach guarantees correctness and is straightforward to implement. Each operation acquires a lock before accessing shared data, ensuring safe execution. However, under high contention, threads may block frequently, limiting scalability.

Lock-Free Queue Design

The lock free queue was implemented using atomic operations and a node based structure. Instead of locks, the queue relies on compare and swap operations to update shared pointers. This allows multiple threads to operate concurrently without blocking. During development, the implementation encountered instability due to unsafe memory access. Nodes were being deallocated while still being accessed by other threads, leading to crashes under contention. A temporary fix was applied by deferring memory deallocation to ensure stability during benchmarking. Although lock free designs reduce blocking, they introduce complexity in memory management and correctness.

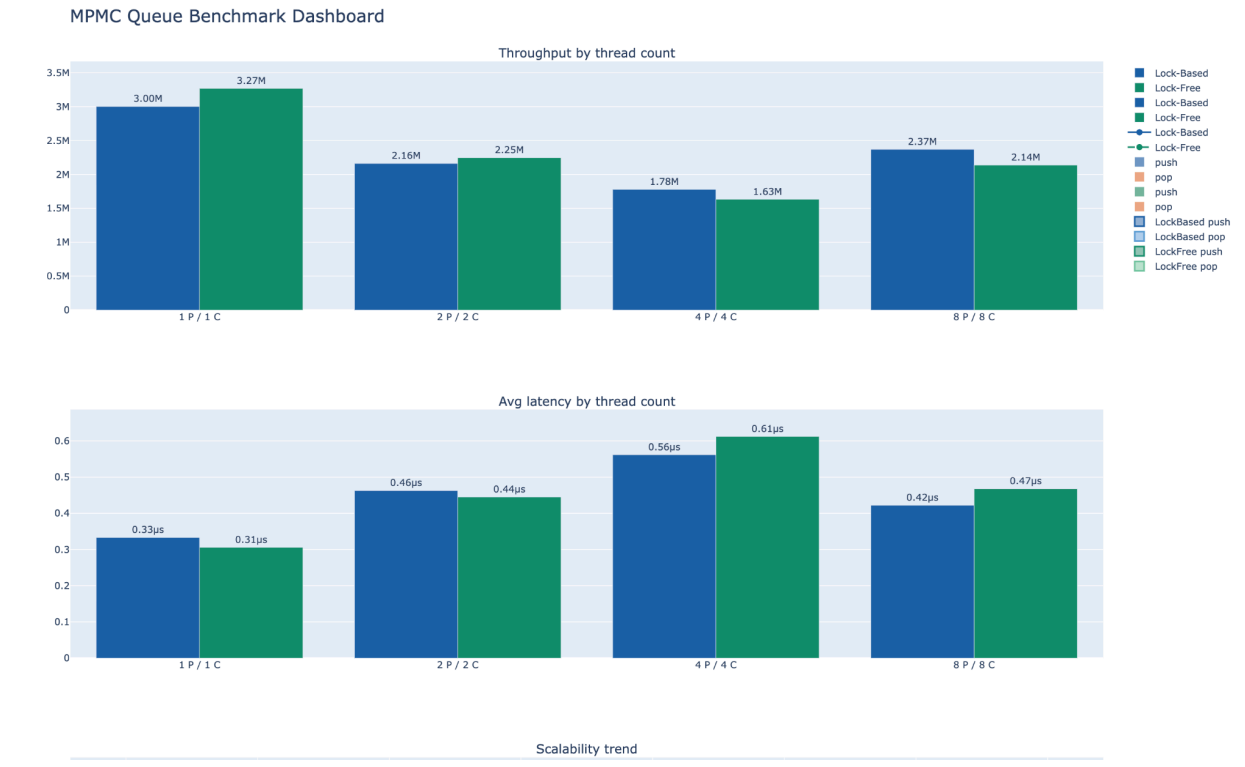
Methodology

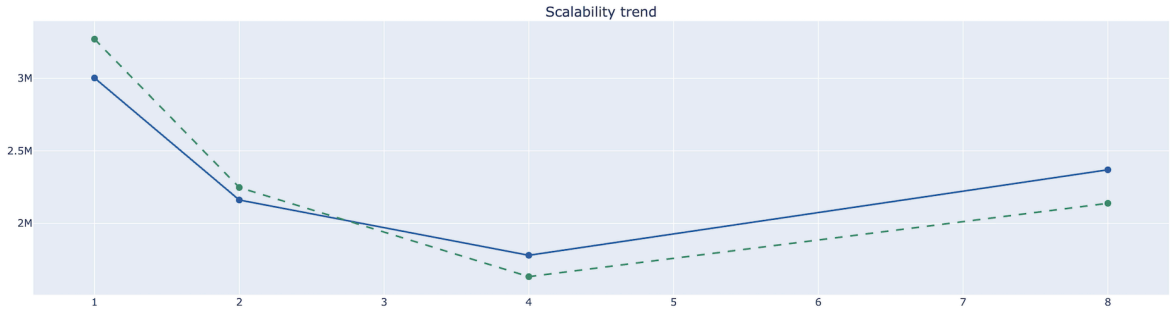
Both queue implementations were evaluated using a benchmarking system designed to simulate concurrent workloads. Multiple producer and consumer threads were used to test performance under different levels of contention. The configurations tested included one producer and one consumer, two producers and two consumers, four producers and four consumers, and eight producers and eight consumers. Each test executed a fixed number of operations and recorded execution time. Performance was measured using throughput, defined as operations per second, and latency, defined as the average time per operation. Timing was collected using high resolution clocks.

Results

The lock based queue demonstrated consistent and stable performance across all test configurations. Throughput ranged from approximately eleven million operations per second in the single thread configuration to around nine to ten million operations per second under higher concurrency. Latency remained low and consistent across all tests. The lock free queue showed lower performance in comparison. In the single thread configuration, throughput was approximately eight million operations per second. Under increased concurrency, performance degraded further. The implementation also experienced a segmentation fault under contention due to unsafe memory access. After applying a temporary fix, the system completed all tests but continued to underperform relative to the lock based queue.

Performance was evaluated under varying levels of concurrency, with results illustrated in Figure 1





As shown in Figure 1, the lock-based implementation consistently achieved higher throughput and lower latency. Conversely, the lock-free implementation demonstrated significant performance degradation as thread counts increased.

Discussion

The results contradict the common assumption that lock free data structures always provide superior performance. While lock free designs eliminate blocking, they introduce overhead from atomic operations and retry loops. This overhead can reduce performance, especially under moderate levels of contention. The instability observed in the lock free implementation highlights the complexity of concurrent memory management. The segmentation fault was caused by a use after free condition, where memory was deallocated while still in use by another thread. This demonstrates the importance of safe memory management strategies in lock free systems. The lock based queue, although simpler, provided reliable and predictable performance. Its use of mutual exclusion ensured safe memory access without the risks associated with lock free designs.

Conclusion

This study demonstrates that lock free data structures do not automatically guarantee better performance. While they offer theoretical advantages in scalability, their effectiveness depends on correct implementation and system conditions. In this case, the lock based queue outperformed the lock free implementation in both stability and performance. These findings highlight the importance of empirical testing and careful design when working with concurrent systems.

References

Herlihy, M., & Shavit, N. (2012). *The art of multiprocessor programming* (Rev. ed.). Morgan Kaufmann.

Michael, M. M., & Scott, M. L. (1996). Simple, fast, and practical non-blocking and blocking concurrent queue algorithms. In *Proceedings of the fifteenth annual ACM symposium on Principles of distributed computing* (pp. 267–275). https://doi.org/10.1145/248052.248106

Moody, C., & Sutter, H. (2014). *C++ concurrency in action* (2nd ed.). Manning Publications.

Harris, T. L. (2001). A pragmatic implementation of non-blocking linked lists. In *Proceedings of the 15th International Conference on Distributed Computing* (pp. 300–314). Springer. https://doi.org/10.1007/3-540-45414-4_21